

SESSION 4

Django Models & Database Connectivity

Topics Covered: Creating models, Model fields, SQLite to MySQL migration, Migrations: makemigrations, migrate, ORM basics

Activities: Create model: Doctor, Specialization, Connect Django to MySQL using pymysql

Task 1: Create a Django model named Restaurant with fields name (CharField), location (CharField), and rating (FloatField) in a new app called foodiespot.

Explanation & Approach: *Django Models define database table schemas as Python classes using Field types.*

```
# foodiespot/models.py
from django.db import models

class Restaurant(models.Model):
    name = models.CharField(max_length=100)
    location = models.CharField(max_length=200)
    rating = models.FloatField()

    def __str__(self):
        return f"{self.name} - ({self.rating})★"
```

Task 2: Add a Cuisine model to foodiespot with fields: name and description, then set up a ForeignKey in Restaurant linking each restaurant to a cuisine.

Explanation & Approach: *ForeignKey creates a One-to-Many relational database constraint between Cuisine (One) and Restaurant (Many).*

```
# foodiespot/models.py (Updated with Cuisine and ForeignKey)
from django.db import models

class Cuisine(models.Model):
    name = models.CharField(max_length=50)
    description = models.TextField(blank=True, null=True)

    def __str__(self):
        return self.name

class Restaurant(models.Model):
    name = models.CharField(max_length=100)
    location = models.CharField(max_length=200)
    rating = models.FloatField()
    cuisine = models.ForeignKey(Cuisine, on_delete=models.CASCADE,
                               related_name='restaurants', null=True, blank=True)

    def __str__(self):
        return f"{self.name} ({self.cuisine.name if self.cuisine else 'No Cuisine'}) - {self.rating}★"
```

Task 3: Run makemigrations and migrate commands to create database tables, then verify tables in SQLite database.

Explanation & Approach: *makemigrations translates model code into migration SQL instructions; migrate executes them against the database engine.*

```
# Step 1: Generate migration files
python manage.py makemigrations foodiespot

# Output:
# Migrations for 'foodiespot':
#   foodiespot/migrations/0001_initial.py
#   - Create model Cuisine
#   - Create model Restaurant

# Step 2: Apply migrations to create DB tables
python manage.py migrate

# Output:
# Running migrations:
#   Applying foodiespot.0001_initial... OK
```

Task 4: Change database settings to connect to MySQL using pymysql, and update DATABASES in settings.py.

Explanation & Approach: *pymysql allows Django to interface with MySQL servers using standard MySQL client protocols in Python environments.*

```
# foodiehub_project/__init__.py
import pymysql
pymysql.install_as_MySQLdb()

# foodiehub_project/settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'foodiespot_db',
        'USER': 'root',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

Task 5: Use Django ORM to add three Restaurant objects and two Cuisine objects, then write a query to get all restaurants with rating above 4.0 and print their names.

Explanation & Approach: *Django ORM abstraction allows Python code to perform SQL CRUD operations without writing raw SQL queries.*

```
# Django Shell ORM Script (python manage.py shell)
from foodiespot.models import Restaurant, Cuisine

# Step 1: Create Cuisine objects
chinese = Cuisine.objects.create(name="Chinese", description="Delicious noodles, dim sums,
and spicy manchurian")
italian = Cuisine.objects.create(name="Italian", description="Authentic pizzas and
handcrafted pastas")

# Step 2: Create Restaurant objects
r1 = Restaurant.objects.create(name="Mainland China", location="MG Road, Bangalore",
rating=4.5, cuisine=chinese)
r2 = Restaurant.objects.create(name="Toscano", location="UB City, Bangalore", rating=4.2,
cuisine=italian)
r3 = Restaurant.objects.create(name="Fast Noodle Bar", location="Indiranagar", rating=3.8,
cuisine=chinese)

# Step 3: Query restaurants with rating > 4.0
high_rated = Restaurant.objects.filter(rating__gt=4.0)

print("--- Restaurants Rated Above 4.0 ---")
for r in high_rated:
    print(f"Name: {r.name} | Rating: {r.rating} | Cuisine: {r.cuisine.name}")
```